

Architecture for Exposing Brokerage Trading and Research Capabilities as Agentic AI Tools

A Non-MCP Design for Bring-Your-Own-Agent Consumption of Business Capability APIs

Enterprise Architecture — Digital Platform Engineering

July 30, 2026

Table of Contents

Document Control	3
Executive Summary	4
1 Purpose, Scope, and Audience	5
1.1 Purpose	5
1.2 In Scope	5
1.3 Out of Scope (Phase 2 Candidates)	5
1.4 Audience	5
1.5 Non-Goals	5
2 Business Capability Domain Model	6
2.1 Brokerage Domain Overview	6
2.2 In-Scope Sub-Domains	6
2.3 Why the Domain Boundary Matters for Agentic Exposure	6
3 Design Principles and Constraints	7
3.1 Why Not MCP for This Program	7
3.2 Guiding Principles	7
4 Target Architecture Overview	8
4.1 Actors and Trust Zones	8
4.2 High-Level Architecture	8
4.3 Component Responsibilities	8
5 Exposing Business Capability APIs for LLM Consumption Without MCP	10
5.1 The Core Problem	10
5.2 Tool Manifest and Discovery	10
5.3 Tool Registry	11
5.4 Tool Search Service	12
5.5 Tool Execution Broker	12
5.6 REST/OpenAPI-Native Tool Contract, Not JSON-RPC-over-MCP	12
5.7 Comparison: This Pattern vs. an MCP Server	12
6 Identity and Access: Authenticating the Customer, Not the API	14
6.1 Principle	14

6.2	OAuth 2.1 + OpenID Connect with the Brokerage IdP	14
6.3	Multi-Factor and Step-Up Authentication	14
6.4	Consent and Fine-Grained, Rich Authorization Scopes	14
6.5	Token Types and Sender-Constraining	15
6.6	Dynamic Client Registration for Agent Platforms	15
7	eJWT: Encrypted Identity Propagation Across the DMZ	17
7.1	Who Generates the eJWT, and Why	17
7.2	eJWT Structure: Signed, Then Encrypted (Nested JWT)	17
7.3	eJWT Claims Reference	17
7.4	Key Management and Rotation	18
7.5	Defense in Depth: eJWT + mTLS + Zero Trust	18
8	Sequence Flows	19
8.1	Connect, Discover, Search, and Use	19
8.2	Authentication, API Gateway Access, and eJWT Issuance	19
8.3	Worked Example: Placing an Equity Order	21
9	Trading and Research Tool Catalog (Initial Scope)	23
9.1	Research Tools	23
9.2	Trading Tools	23
9.3	Tool Definition Schema Example: place_equity_order	24
10	Worked Example: Placing a Trade via a Third-Party Agent	26
11	Security Best Practices	28
11.1	Zero Trust and Least Privilege	28
11.2	Perimeter and Transport Security	28
11.3	Anti-Automation, Abuse, and Prompt-Injection Defenses	28
11.4	Human-in-the-Loop and Trade Controls	28
11.5	Auditability and Non-Repudiation	29
11.6	Secrets and Key Management	29
11.7	Data Minimization and Response Shaping	29
11.8	Regulatory Considerations	29
12	Governance and Lifecycle Management	30
13	Non-Functional Requirements	31
14	Roadmap	32
Appendix A	— Glossary	33
Appendix B	— eJWT Claims Quick Reference	33
Appendix C	— References	33

Document Control

Field	Value
Document status	Draft for Architecture Review Board
Classification	Internal — Confidential
Primary domains in scope	Trading, Research
Domains deferred to Phase 2	Money Movement, Account Management, Customer Management
Audience	Chief Architect, Enterprise Architecture, Security Architecture, Platform Engineering
Related standards	OAuth 2.1, OpenID Connect, FAPI 2.0, RFC 8693, RFC 9396, RFC 7591, NIST SP 800-63B

Confidentiality notice. This document describes internal system design, security controls, and key-management practices for a regulated brokerage platform. Distribute only to authorized architecture and security personnel.

Executive Summary

Brokerage customers increasingly want to delegate research and trading tasks to AI agents that they choose and operate themselves — a general-purpose assistant such as Claude or ChatGPT, an IDE-embedded agent, or a custom application built by a fintech partner. This creates a new access channel that sits outside the firm’s traditional web and mobile applications: the **bring-your-own-agent (BYOA)** channel.

This document specifies the target architecture for exposing a defined subset of brokerage **business capability APIs** — initially the **Trading** and **Research** sub-domains — as a catalog of discrete, well-described **tools** that any third-party AI agent can call over standard HTTPS/REST, without the brokerage hosting or operating a Model Context Protocol (MCP) server. Money Movement, Account Management, and Customer Management remain out of scope for this phase and are addressed briefly in the roadmap.

The design is built around four ideas that recur throughout the document:

1. **Never hand the agent the keys to the API.** The agent never receives the customer’s brokerage password, never talks directly to an internal domain API, and never receives a token that is valid inside the internal network. It receives only short-lived, narrowly scoped, sender-constrained OAuth tokens issued by the brokerage’s own identity provider after the customer authenticates with brokerage credentials plus a second factor.
2. **Tools are a product, not a side effect of the API.** A dedicated **Tool Registry**, **Tool Search Service**, and **Tool Execution Broker** sit between the public edge and the internal domain APIs. They translate business capability endpoints into agent-consumable, versioned, schema-validated tool contracts, independent of any particular agent framework or protocol.
3. **Identity is re-minted, not forwarded, at the trust boundary.** The externally issued OAuth access token is never passed into the internal network. At the DMZ boundary, an internal Security Token Service exchanges it for a short-lived, encrypted identity assertion — an **eJWT** (signed, then encrypted JSON Web Token) — that is the only form of identity internal Trading and Research services trust.
4. **High-consequence actions stay human-in-the-loop.** Placing or modifying an order always flows through a pre-trade review step, explicit customer confirmation (unless the customer has pre-authorized unattended trading within firm-approved limits), and a risk/compliance interceptor, regardless of which agent platform initiated the request.

The document includes four architecture diagrams (high-level component view, connect/discover/search/use sequence, authentication and eJWT-issuance sequence, and a full order-placement sequence), a tool catalog for the initial Trading and Research scope modeled on patterns already observed in production agentic-trading offerings, worked request/response examples for placing an equity order, and a security and governance framework suitable for review by a chief architect and a security architecture board.

1 Purpose, Scope, and Audience

1.1 Purpose

Define a target architecture that lets an external, customer-owned AI agent discover and invoke a governed set of brokerage capabilities — read-only research data and trade order management — through a secure, non-MCP tool interface, while preserving the same authentication strength, authorization granularity, and auditability the firm already requires for its web and mobile channels.

1.2 In Scope

- **Research sub-domain:** quotes, fundamentals, historical price bars, technical indicators, earnings data, index data, watchlists, symbol search, market scanners.
- **Trading sub-domain:** account and portfolio summaries, positions, order history, pre-trade order review, equity and options order placement, order cancellation.
- The edge, identity, and tool-exposure architecture that makes the above safely consumable by third-party agents.

1.3 Out of Scope (Phase 2 Candidates)

- **Money Movement** (ACH transfers, wires, instant funding).
- **Account Management** (account opening, document e-signature, tax elections).
- **Customer Management** (profile changes, beneficiaries, contact information).

These sub-domains carry different risk profiles (funds movement, identity changes) and are deliberately deferred; Section 14 outlines how this architecture extends to them.

1.4 Audience

Chief Architect and Architecture Review Board, Security Architecture, Identity and Access Management, Platform/API Engineering, Trading Technology, and Compliance/Legal stakeholders reviewing agentic-channel risk.

1.5 Non-Goals

This document does not select a specific API gateway or IdP vendor, does not define the internal Trading/Research microservice decomposition beyond what is needed to reason about identity propagation, and does not restate FINRA/SEC recordkeeping rules in full — it identifies where they apply.

2 Business Capability Domain Model

2.1 Brokerage Domain Overview

The brokerage platform’s business capabilities are organized into five sub-domains, each owning its own APIs, data, and bounded context:

Sub-domain	Representative capabilities	Phase
Trading	Order entry, order management, positions, portfolio valuation	Phase 1 (this document)
Research	Quotes, fundamentals, historicals, technical indicators, news/earnings, scanners	Phase 1 (this document)
Money Movement	ACH/wire transfers, instant deposits, bank linking	Phase 2
Account Management	Account opening, statements, tax documents, entitlements	Phase 2
Customer Management	Identity/profile, preferences, beneficiaries	Phase 2

2.2 In-Scope Sub-Domains

Trading and **Research** are prioritized because they are (a) the capabilities customers most want an agent to use in natural language — “what’s my buying power,” “how has this stock performed,” “buy \$500 of X” — and (b) already exposed as governed, versioned business capability APIs behind the firm’s existing internal API gateway, which minimizes new back-end build and concentrates new investment in the agent-facing exposure and security layers described in this document.

2.3 Why the Domain Boundary Matters for Agentic Exposure

Each sub-domain becomes an independent **tool namespace** with its own OAuth scopes, risk tier, and rate-limit policy (Section 5.3, Section 6.4). This means Money Movement can be added later as its own namespace, with its own step-up requirements, without re-architecting the Trading and Research exposure already in production — a direct consequence of the layered design in Section 4.

3 Design Principles and Constraints

3.1 Why Not MCP for This Program

MCP is a strong, increasingly standard protocol for connecting a single AI runtime to a set of tools, and it is a reasonable choice for many internal or developer-tool integrations — Robinhood’s own agentic trading offering, for example, is built on a hosted MCP server (<https://agent.robinhood.com/mcp/trading>) that customers add to Claude, ChatGPT, Cursor, and similar clients. This document intentionally specifies a **non-MCP** architecture, for reasons a review board should understand explicitly rather than infer:

1. **Protocol neutrality.** MCP is one of several competing and complementary agent-integration patterns (MCP, OpenAI-style Actions/plugin manifests, direct function calling against a documented REST API, the emerging Agentic Commerce Protocol for payment-scoped delegation). A REST/OpenAPI-based tool contract is consumable by *any* agent runtime — MCP-based or not — without the firm depending on a single protocol’s transport, session, and hosting model.
2. **Operational control of the trust boundary.** An MCP server is commonly deployed as a long-lived, stateful process that proxies tool calls; operating one inside or at the edge of a regulated environment introduces a new class of infrastructure (session state, SSE/streamable-HTTP transport, MCP-specific auth extensions) that has to be hardened, scaled, and audited independently of the firm’s existing, mature API gateway and OAuth stack. A stateless REST tool-execution broker reuses that existing, audited infrastructure.
3. **Fine-grained, brokerage-specific authorization.** Financial-grade authorization patterns — Rich Authorization Requests (RFC 9396) with dollar limits and symbol restrictions, FAPI 2.0 sender-constrained tokens, step-up re-authentication mid-session for high-risk actions — are mature in the OAuth/OIDC ecosystem and are what this design builds on directly, rather than layering financial-grade semantics on top of a newer, still-evolving protocol’s auth model.
4. **Auditability and non-repudiation.** Every tool call in this design is a discrete, signed HTTPS request/response pair that lands in the firm’s existing API gateway logs and audit event bus in the same shape as every other regulated API call, which simplifies FINRA/SEC recordkeeping and surveillance tooling that already expects request/response, not a persistent bidirectional session.

This is a deliberate platform decision, not a rejection of MCP’s technical merits. Section 14 notes how an MCP adapter could later be layered *on top of* this same tool registry/broker without changing the underlying security model, if the firm later decides to support MCP-native agent platforms as an additional transport.

3.2 Guiding Principles

- **Least privilege by default.** Every tool call carries the minimum scope and data needed; nothing is returned “just in case.”
- **Customer-present, customer-controlled.** The customer authenticates directly with the brokerage IdP — never through the agent — and explicitly consents to what the agent may see and do.
- **Stateless, idempotent tool execution.** Every mutating tool call carries an idempotency key; no tool call depends on server-side conversational session state.
- **Defense in depth.** Perimeter controls (WAF, rate limiting), identity controls (OAuth, MFA, DPOP), and application controls (risk engine, human confirmation) are independent and redundant.
- **Everything is versioned and revocable.** Tool contracts, consent grants, and tokens are all independently versionable and revocable without redeploying the domain APIs.

4 Target Architecture Overview

4.1 Actors and Trust Zones

Zone	Operated by	Trust level	Contents
Zone A — Bring-your-own agent	Customer / third-party AI platform	Untrusted	Customer, agent platform (Claude, ChatGPT, custom app), agent orchestrator, HTTPS tool-calling client
Zone B — Public edge / DMZ	Brokerage	Perimeter-trusted	WAF, edge API gateway, OAuth 2.1/OIDC identity provider, tool-manifest discovery endpoint
Zone C — Agent Tool Gateway	Brokerage	Semi-trusted (application tier)	Tool Registry, Tool Search Service, Tool Execution Broker, Policy/Risk Engine, internal Security Token Service (eJWT issuer)
Zone D — Internal trust zone	Brokerage	Fully trusted	Trading Domain API, Research Domain API, Order Management System, Market Data Platform, Position/Account Ledger, Audit/Event Bus

4.2 High-Level Architecture

Figure 1 shows the complete component view. The customer’s agent never crosses into Zone C or Zone D directly; every request is re-authenticated and re-authorized at each boundary crossing, and identity is re-minted (not simply forwarded) when crossing from Zone C into Zone D.

4.3 Component Responsibilities

Component	Zone	Responsibility
WAF / bot management	B	TLS termination, request inspection, bot and anti-automation defense, DDoS mitigation

Component	Zone	Responsibility
Edge API Gateway	B	Token validation/introspection, routing, quota and rate-limit enforcement, request/response logging
Customer IdP (OAuth 2.1 / OIDC)	B	Brokerage credential authentication, MFA/step-up, consent capture, scoped token issuance
Tool Manifest Discovery	B	Publishes the signed, versioned catalog of tools an agent can request
Tool Registry	C	System of record for tool definitions: schema, backend mapping, required scope, risk tier, version, lifecycle state
Tool Search Service	C	Keyword and semantic search/ranking over the registry so an agent can find the right tool from a natural-language query
Tool Execution Broker	C	Validates requests against JSON Schema, enforces idempotency, orchestrates calls to internal APIs, shapes responses
Policy, Entitlement & Risk Engine	C	Scope/entitlement checks, pre-trade limits, suitability checks, anomaly/fraud detection
Internal STS (eJWT issuer)	C	Mints the encrypted, signed internal identity assertion (eJWT) trusted by Zone D services
Trading / Research Domain APIs	D	System-of-record business capability APIs (unchanged by this program; consumed, not re-platformed)
Order Management System	D	Order lifecycle, execution venue routing, fills
Audit & Event Bus	D	Immutable event log for every tool invocation and order lifecycle event

Figure 1 — High-Level Architecture: Agentic Tool Exposure for Trading & Research (Non-MCP)

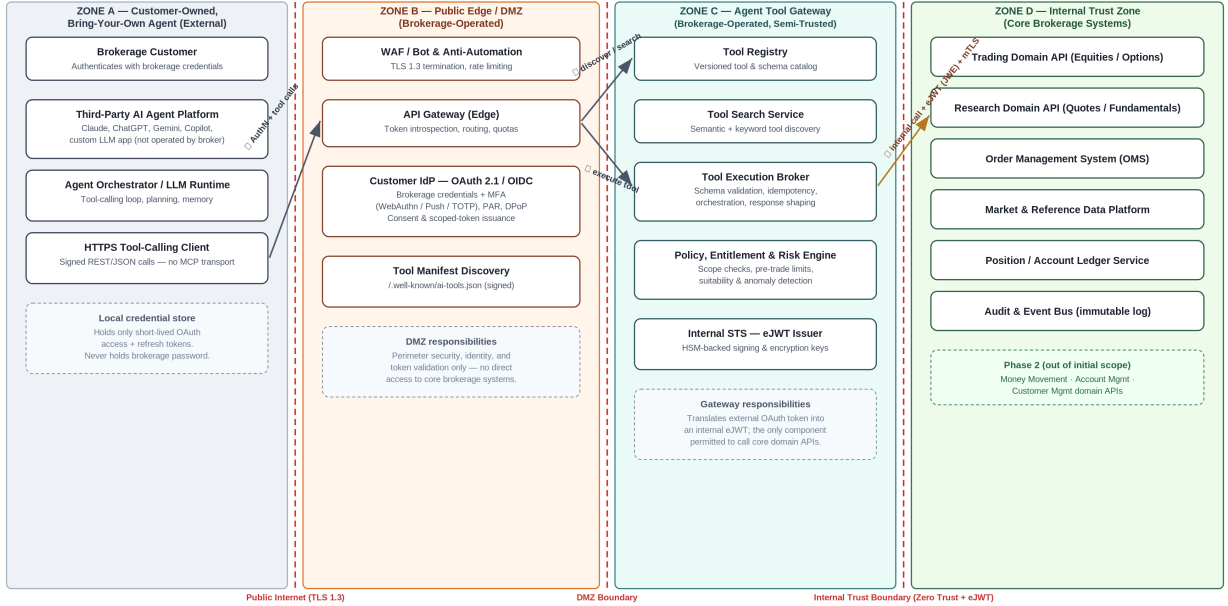


Figure 1: High-level architecture: agentic tool exposure for Trading and Research, showing the four trust zones and the boundary controls between them.

5 Exposing Business Capability APIs for LLM Consumption Without MCP

5.1 The Core Problem

Business capability APIs (Trading, Research) are designed for deterministic, schema-aware client applications — the firm’s own mobile app, a market-data vendor, an internal service. An LLM-driven agent is a different kind of client: it composes calls dynamically based on natural language, benefits from rich human-readable descriptions and examples, needs to discover *which* capability to call before it can call it, and must never be trusted with more access than the specific action it is about to take requires. Simply publishing the existing OpenAPI specification and pointing an agent at it is insufficient — it exposes the full surface area of the internal API (including operations never meant for external, autonomous callers), has no concept of a “tool,” and has no mechanism to scope a token down to exactly the operation being attempted.

The answer is an intermediate **Agent Tool Gateway** layer (Zone C) that does three things a raw API does not: **registers** a curated, descriptive subset of capabilities as tools; makes those tools **discoverable and searchable** by an agent; and **brokers execution** with schema validation, entitlement checks, and identity translation — all without requiring the brokerage to run an MCP server or the agent to speak anything other than HTTPS and JSON.

5.2 Tool Manifest and Discovery

Analogous to the plugin/actions manifests used by several agent platforms, the brokerage publishes a signed, versioned **tool manifest** at a well-known, unauthenticated URL:

GET <https://agent-api.brokerage.com/.well-known/ai-tools.json>

The manifest is deliberately *not* an MCP tools/list response and is *not* served over an MCP

transport. It is a plain, cacheable JSON document, signed with the firm’s manifest-signing key so an agent (or its platform operator) can verify it has not been tampered with in transit:

```
{
  "manifest_version": "2026-07-01",
  "issuer": "https://agent-api.brokerage.com",
  "oauth": {
    "authorization_endpoint": "https://id.brokerage.com/oauth2/authorize",
    "pushed_authorization_request_endpoint": "https://id.brokerage.com/oauth2/par",
    "token_endpoint": "https://id.brokerage.com/oauth2/token",
    "jwks_uri": "https://id.brokerage.com/.well-known/jwks.json",
    "scopes_supported": [
      "research:read",
      "trading:read",
      "trading:review",
      "trading:execute:equity",
      "trading:execute:option",
      "watchlist:write"
    ]
  },
  "tool_catalog_endpoint": "https://agent-api.brokerage.com/v1/tools",
  "tool_search_endpoint": "https://agent-api.brokerage.com/v1/tools/search",
  "tool_execute_endpoint": "https://agent-api.brokerage.com/v1/tools/execute",
  "signature": "base64url(JWS over the manifest body, key id mfst-2026-07)"
}
```

5.3 Tool Registry

The Tool Registry is the system of record for every tool the firm exposes to agents. Each entry binds a human- and LLM-readable tool description to an internal backend operation, an OAuth scope, a risk tier, and a JSON Schema contract. A simplified record:

```
{
  "tool_id": "place_equity_order",
  "version": "1.3.0",
  "domain": "trading",
  "status": "GA",
  "description": "Place a new equity (stock) order in the customer's brokerage account. Always",
  "required_scope": "trading:execute:equity",
  "risk_tier": "HIGH",
  "idempotent": true,
  "backend": {
    "system": "trading-domain-api",
    "operation": "POST /internal/v3/orders/equity"
  },
  "input_schema": "schemas/place_equity_order.request.json",
  "output_schema": "schemas/place_equity_order.response.json",
  "rate_limit": "10 / minute / customer",
  "deprecates": null
}
```

Registry entries are versioned independently of the underlying domain API; a breaking change to a tool’s schema ships as a new tool_id/version with a deprecation window on the old one, so agent integrations do not break silently (Section 12).

5.4 Tool Search Service

Because the full catalog can span dozens of tools across Trading and Research (and grow further once Money Movement and Account Management are added), agents are expected to **search**, not enumerate, the catalog:

```
POST /v1/tools/search
```

```
Authorization: DPoP <access_token>
```

```
Content-Type: application/json
```

```
{ "query": "place a stock order", "domain": "trading", "max_results": 5 }
```

The search service combines keyword matching over tool names/descriptions with a semantic embedding index (so “buy some shares” resolves to `place_equity_order`, and “what’s this stock worth” resolves to `get_equity_quotes` / `get_equity_fundamentals`), and — critically — **filters results to tools the caller’s current access token scope actually permits**, so an agent never even sees a tool it is not entitled to invoke.

5.5 Tool Execution Broker

All mutating and data-returning tool calls flow through a single execution endpoint:

```
POST /v1/tools/execute
```

```
Authorization: DPoP <access_token>
```

```
Idempotency-Key: 6e2f9e2a-...
```

```
Content-Type: application/json
```

```
{ "tool_id": "get_equity_quotes", "version": "1.1.0", "arguments": { "symbols": ["AAPL"] } }
```

The broker performs, in order: (1) token validation and scope check against the tool’s `required_scope`; (2) JSON Schema validation of arguments against the tool’s `input_schema`; (3) idempotency-key deduplication for mutating calls; (4) a call to the Policy/Risk Engine for high-risk tools; (5) identity translation to an eJWT (Section 7) and the internal call to the domain API; (6) response shaping against the tool’s `output_schema`, stripping any internal fields not part of the public tool contract; and (7) an audit event emission. Only the broker — never the agent — holds credentials or network access to Zone D.

5.6 REST/OpenAPI-Native Tool Contract, Not JSON-RPC-over-MCP

Each tool’s `input_schema/output_schema` is standard JSON Schema, and the catalog itself can be rendered as an OpenAPI 3.1 document (`GET /v1/openapi.json`) for agent frameworks that prefer to import an OpenAPI spec and generate function-calling definitions from it automatically. This gives the firm two equally valid on-ramps for agent platforms: consume the lightweight tool manifest/search/execute endpoints directly, or import the OpenAPI document into whatever function-calling mechanism the agent platform natively supports — without the firm building or operating protocol-specific server infrastructure for each.

5.7 Comparison: This Pattern vs. an MCP Server

Dimension	Non-MCP Tool Gateway (this design)	Hosted MCP Server
Transport	Standard HTTPS/REST + JSON	MCP-specific (stdio, SSE, or streamable HTTP)

Dimension	Non-MCP Tool Gateway (this design)	Hosted MCP Server
Client compatibility	Any HTTP-capable agent/orchestrator	Requires an MCP-capable client
Session model	Stateless per-call, idempotency-key based	Often session/connection-oriented
Auth model	Firm’s existing OAuth 2.1/FAPI stack, unmodified	OAuth 2.1 required by MCP spec, but as an MCP-specific authorization extension the firm must additionally support
Discovery	Signed manifest + registry search API	MCP tools/list over an active session
Infra reuse	Existing API gateway, WAF, IdP	New MCP server runtime to build, hard, and operate
Best fit	Programs prioritizing protocol neutrality and reuse of existing regulated API infrastructure	Programs standardizing on MCP-native agent platforms as the primary channel

6 Identity and Access: Authenticating the Customer, Not the API

6.1 Principle

The agent is never issued a static API key, never stores the customer’s brokerage password, and never talks to an internal domain API directly. What the agent receives is a short-lived, narrowly scoped, sender-constrained **OAuth access token**, obtained only after the *customer* — not the agent, not the agent platform vendor — completes an interactive authentication and consent step directly with the brokerage’s identity provider.

6.2 OAuth 2.1 + OpenID Connect with the Brokerage IdP

Authorization uses the OAuth 2.1 **authorization code flow with PKCE**, hardened per the FAPI 2.0 security profile:

- **Pushed Authorization Requests (PAR)** — the agent submits the authorization request parameters directly to the IdP over a back-channel call before redirecting the customer, so sensitive parameters (requested scopes, redirect URI) never transit the customer’s browser as a URL.
- **PKCE** (code_challenge/code_verifier) is mandatory for every client, public or confidential.
- The actual login page is rendered and hosted **only** by the brokerage IdP; the agent platform is redirected to it and never sees the credential-entry surface.
- **Sender-constrained tokens** via DPoP (or mutual TLS where the agent platform supports client certificates) bind the issued access token to a specific key pair held by the agent’s client, so a stolen bearer token alone is not sufficient to replay a call.

6.3 Multi-Factor and Step-Up Authentication

Brokerage username/password alone is never sufficient to authorize an agent connection:

- **Primary authentication:** brokerage username + password (or passkey), authenticated attempt strength recorded as an Authenticator Assurance Level (AAL) per NIST SP 800-63B.
- **Mandatory second factor** at connection time: WebAuthn/passkey, push notification to the firm’s mobile app, or TOTP — the same MFA stack already used for the firm’s web/mobile login, reused rather than duplicated.
- **Step-up re-authentication** is triggered mid-session, independent of token expiry, when: a trade exceeds a customer- or firm-configured dollar threshold; the agent requests a scope broader than previously granted; the request originates from a new device/IP/agent-platform fingerprint; or elapsed time since the last strong authentication exceeds the policy for the requested risk tier (see Section 8.3 for a worked step-up example).

6.4 Consent and Fine-Grained, Rich Authorization Scopes

At consent time, the customer sees — in plain language, rendered by the brokerage IdP, never by the agent — exactly what is being requested. Scopes are deliberately granular rather than a single “trading” scope:

Scope	Grants
research:read	Read-only market data, fundamentals, historicals, indicators
trading:read	Read-only accounts, portfolio, positions, order history
trading:review	Pre-trade order simulation (review_*_order) — no order is placed
trading:execute:equity	Place/cancel equity orders, optionally bounded by a per-order or daily dollar cap
trading:execute:option	Place/cancel options orders (requires the account to already hold options trading approval)
watchlist:write	Create/modify watchlists and scans

Where a dollar or symbol limit applies, the design uses **Rich Authorization Requests (RFC 9396)** so the grant itself — not just a downstream policy table — encodes the limit, e.g. `{"type":"trading:execute:equity","limits":{"max_order_value_usd":2000,"daily_cap_usd":5000}}`. The IdP renders these limits on the consent screen and embeds them as claims in the issued token, so the Policy/Risk Engine (Section 5.5) can enforce them without a separate out-of-band lookup.

Consent is: **explicit** (opt-in per scope, not bundled), **visible** (customer can view active grants and revoke them at any time from brokerage account settings, immediately invalidating outstanding tokens for that grant), and **time-bounded** (grants expire and must be renewed; refresh tokens are themselves capped and rotated).

6.5 Token Types and Sender-Constraining

Token	Purpose	Typical TTL	Binding
Authorization code	One-time exchange for tokens	≤ 60 seconds	PKCE-bound
Access token	Authorizes tool-execute calls	≤ 15 minutes	DPoP- or mTLS-bound
Refresh token	Silent renewal of access token	Policy-defined, rotated on use	Bound to original client + device
ID token	Proves customer identity to the agent platform (display name only; no account data)	Single use at login	Signed JWT

Short access-token TTLs mean a leaked token has a narrow blast radius; the trade-off (more frequent silent refresh) is handled transparently by the agent platform's OAuth client, not by the customer.

6.6 Dynamic Client Registration for Agent Platforms

Rather than the firm hand-provisioning OAuth client credentials for every possible third-party agent platform, agent platforms register as OAuth clients using **RFC 7591 Dynamic Client Registration** against a firm-curated allow-list — the platform's redirect URIs, software statement, and (where supported) client attestation are validated before a `client_id` is issued.

Unregistered or unattested clients are rejected at the IdP before a customer ever sees a login prompt, which is the first line of defense against a malicious “agent” impersonating a legitimate one.

7 eJWT: Encrypted Identity Propagation Across the DMZ

This is the mechanism that answers the question, “how do we give the agent access to the customer’s data without ever exposing the internal API surface or internal identity model?” The externally issued OAuth access token (Section 6) is valid **only** at the edge (Zone B). It is never forwarded into Zone C or Zone D. Instead, the edge exchanges it for an internal-only, encrypted identity assertion.

7.1 Who Generates the eJWT, and Why

The **eJWT is minted by the brokerage’s internal Security Token Service (STS)** — a purpose-built, HSM-backed issuer that lives inside Zone C, is reachable only from the Edge API Gateway and the Tool Execution Broker, and is never reachable from the public internet. The flow is a **token exchange**, modeled on RFC 8693 (OAuth 2.0 Token Exchange):

1. The Edge API Gateway validates the external OAuth access token (signature via the public IdP’s JWKS, expiry, DPoP proof-of-possession).
2. The gateway calls the internal STS with the validated token’s claims (subject, scopes, AAL, client/device attributes) as a `subject_token`.
3. The STS — which is the *only* component in the architecture permitted to mint identity trusted inside Zone D — issues a new, internal-only token: the eJWT.
4. The eJWT, not the original OAuth token, is what is forwarded (as header `X-Brokerage-Identity`) to Trading and Research domain services.

This means a compromised or overly long-lived external token can never be replayed directly against an internal service: internal services trust only eJWTs signed by the internal STS, which itself only mints an eJWT after independently validating the external token and applying its own, additional risk checks (device reputation, session anomaly score) that are not visible to, or influenced by, the external IdP or the agent.

7.2 eJWT Structure: Signed, Then Encrypted (Nested JWT)

The eJWT is a **nested JWT**: a JWS (signed JWT) containing the identity claims, wrapped inside a JWE (encrypted JWT) so its contents are opaque even to intermediate infrastructure that merely routes the request:

```
eJWT = JWE( JWS( claims, sig_key = STS_private_signing_key ),
            enc_key = target_service_public_encryption_key )
```

- **Signing** uses an asymmetric key pair (ES384) held in the STS’s HSM; downstream services verify the signature using the STS’s published internal JWKS (<https://sts.internal.brokerage.net/.well-known/jwks.json>, reachable only inside Zone C/D).
- **Encryption** uses RSA-OAEP-256 / A256GCM (JWE) with the receiving service’s public key, so only the intended Trading or Research service (holding the matching private key) can decrypt the claims — a gateway or logging pipeline that captures the header in transit sees only ciphertext.
- **Short TTL**: 60–120 seconds, re-minted per tool-execution call (not reused across calls), which bounds replay risk inside the internal network to a very small window in addition to the transport-level mTLS protection described below.

7.3 eJWT Claims Reference

Claim	Description
iss	https://sts.internal.brokerage.net
sub	Brokerage customer ID (internal identifier, never the account number)
act	Acting-party claim identifying the agent platform (client_id from Zone A registration) — supports the OAuth “actor” delegation pattern so downstream systems can distinguish “customer, via Agent X” from a direct customer session
accounts	Authorized account ID(s) this request may touch (e.g., only the customer’s Agentic account)
scope	Scopes carried over from the original OAuth grant, re-validated by the STS
limits	Dollar/quantity limits from the RFC 9396 rich authorization grant, if any
aal	Authenticator Assurance Level achieved at login/step-up (AAL2, AAL3)
risk_score	STS-computed session risk score (device reputation, velocity, anomaly signals)
idempotency_key	Propagated from the originating tool-execute request
exp / iat / jti	Short expiry, issued-at, and a unique token ID logged for replay detection

7.4 Key Management and Rotation

- All STS signing and encryption keys are generated and held in a **Hardware Security Module (HSM)**; private keys never leave the HSM boundary.
- Keys rotate on a defined schedule (signing keys quarterly, encryption keys monthly, or immediately on suspected compromise), with overlapping validity windows published via JWKS kid values so in-flight tokens are not invalidated mid-rotation.
- Each internal consuming service publishes its own encryption public key and rotates it independently; the STS resolves the correct target key per call based on the destination service identity.

7.5 Defense in Depth: eJWT + mTLS + Zero Trust

The eJWT is one of two independent controls on every internal call, not a replacement for network-level security:

- **Transport:** every hop inside Zone C and Zone D uses **mutual TLS** with short-lived workload certificates (SPIFFE/SPIRE-issued identities are a good fit here), so a service will not even accept a connection from an unrecognized workload identity, independent of what token it presents.
- **Identity:** the eJWT proves *which customer, via which agent, with what authority* — independent of *which workload* is calling, which mTLS proves.
- A request must pass **both** checks. A compromised internal workload without a valid, freshly minted eJWT still cannot act on a customer’s behalf; a stolen eJWT presented from an unrecognized workload identity is still rejected at the mTLS layer.

8 Sequence Flows

8.1 Connect, Discover, Search, and Use

Figure 2 shows the complete flow from the moment a customer asks their own agent to connect through the first successful tool call, tying together the discovery mechanism from Section 5 and the authentication mechanism from Section 6.

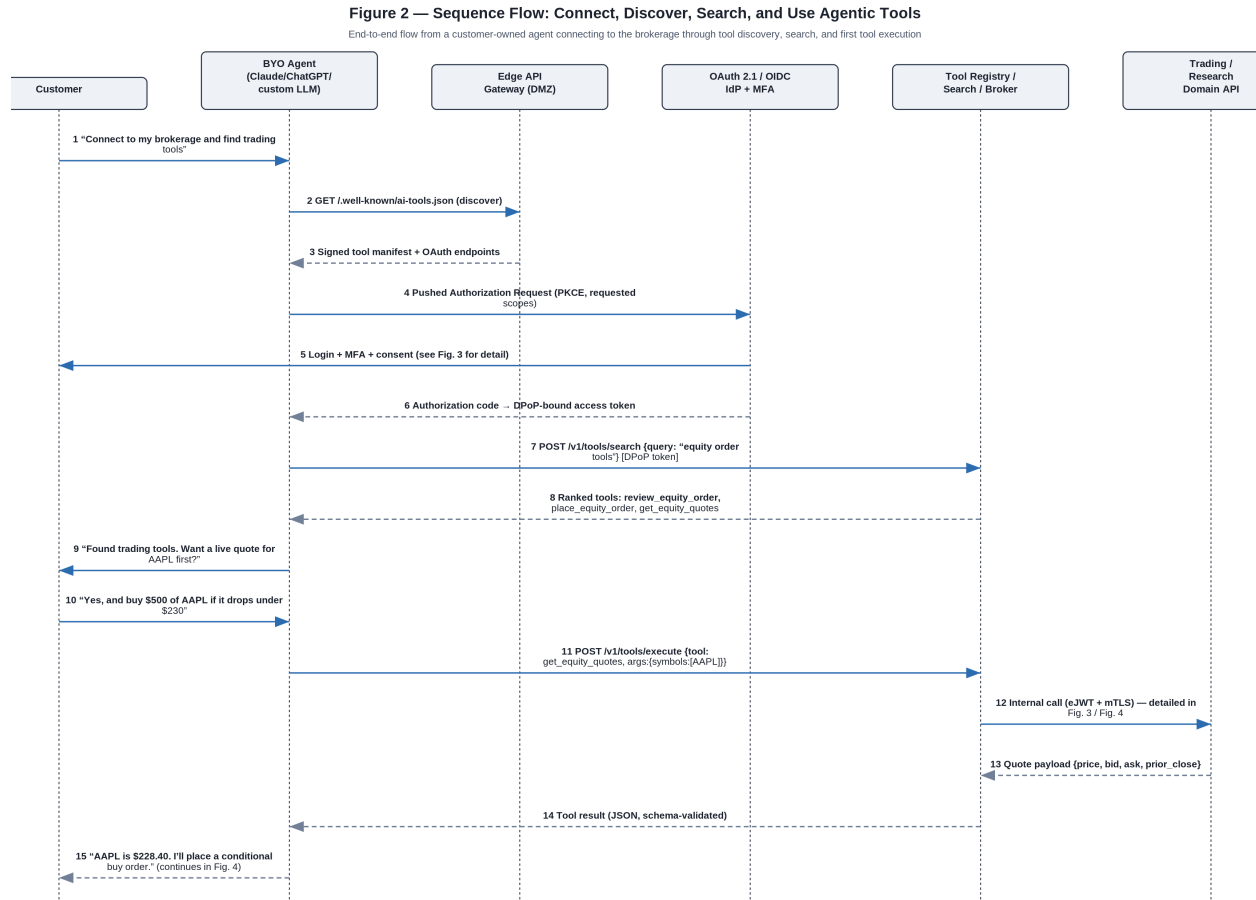


Figure 2: Sequence flow for a bring-your-own agent connecting to the brokerage, discovering the tool manifest, authenticating the customer, searching the tool catalog, and executing a first (read-only) tool call.

Key properties visible in this flow: the agent never sees the login form (step 5 happens directly between the IdP and the customer’s own browser/device); the tool search step (7–8) returns only tools the granted scope permits; and every subsequent tool execution (11) re-uses the same short-lived access token until it expires or a step-up is triggered.

8.2 Authentication, API Gateway Access, and eJWT Issuance

Figure 3 expands the authentication portion of Figure 2 into full detail, and is the diagram that answers “how does a user authenticate to the API gateway and get access to use the tools, and how is the eJWT generated and propagated.”

Steps 1–12 occur entirely between the agent, the edge gateway, the public-facing OAuth/OIDC IdP, and the customer’s own browser or device — this is standard, FAPI-

Figure 3 — Sequence Flow: Customer Authentication, API Gateway Access, and eJWT Issuance

OAuth 2.1 + PKCE + step-up MFA at the edge; internal STS mints an encrypted JWT (eJWT) that is the only identity token trusted inside the DMZ

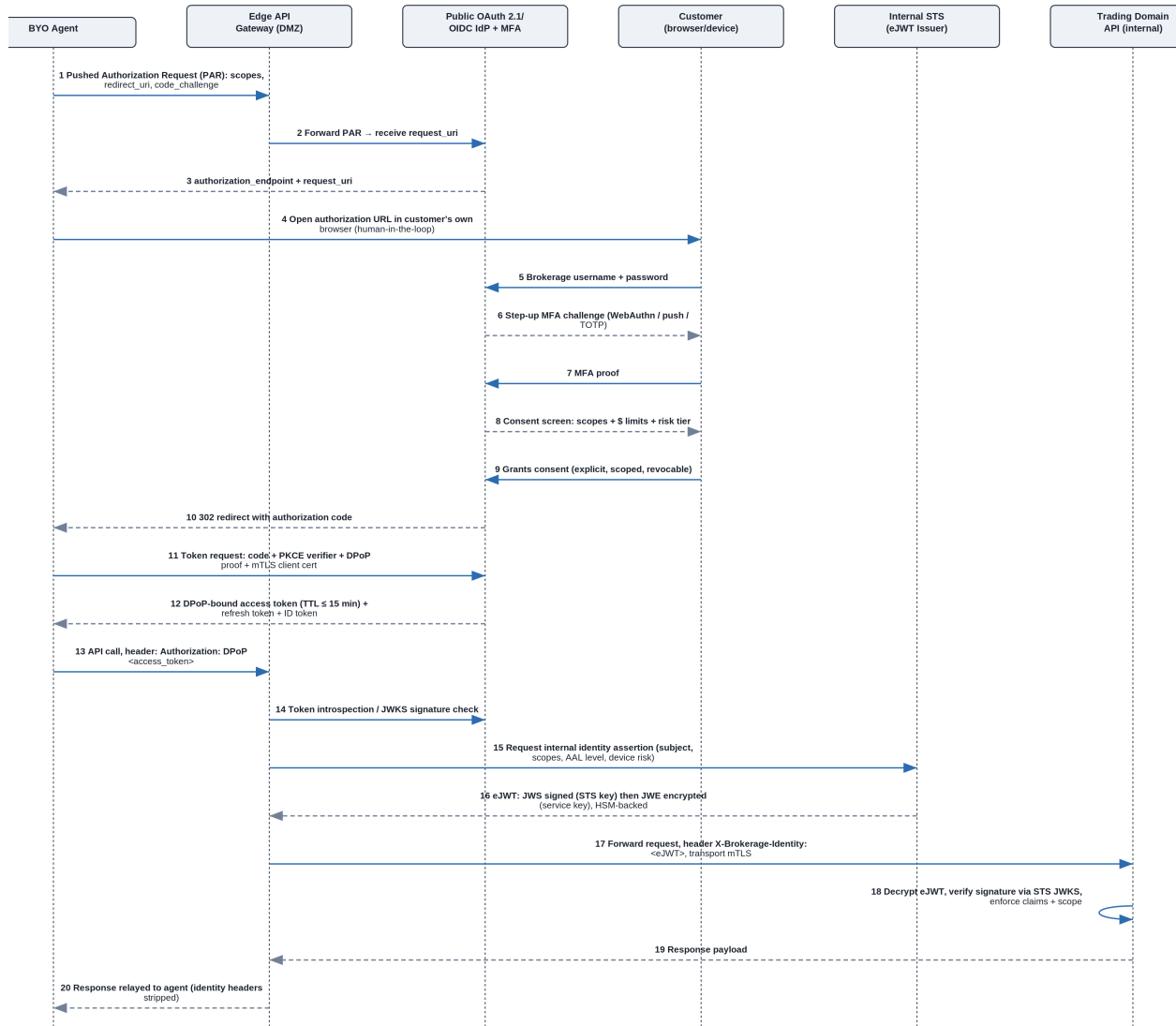


Figure 3: Detailed sequence: OAuth 2.1 + PKCE + step-up MFA at the public edge, followed by the internal STS minting an eJWT that is the only identity token trusted by internal Trading/Research services.

hardened OAuth 2.1 and never touches internal systems. Steps 13-20 are internal: the edge gateway independently validates the external token (14), requests an internal identity assertion from the STS (15), receives the eJWT (16), and only then forwards the call into Zone D (17) — at which point the receiving domain service independently decrypts and verifies the eJWT before doing any work (18).

8.3 Worked Example: Placing an Equity Order

Figure 4 traces a full trade lifecycle, including the mandatory pre-trade review, human confirmation gate, and a second, fresh eJWT minted specifically for the placement call (illustrating the step-up-on-value-threshold pattern from Section 6.3).

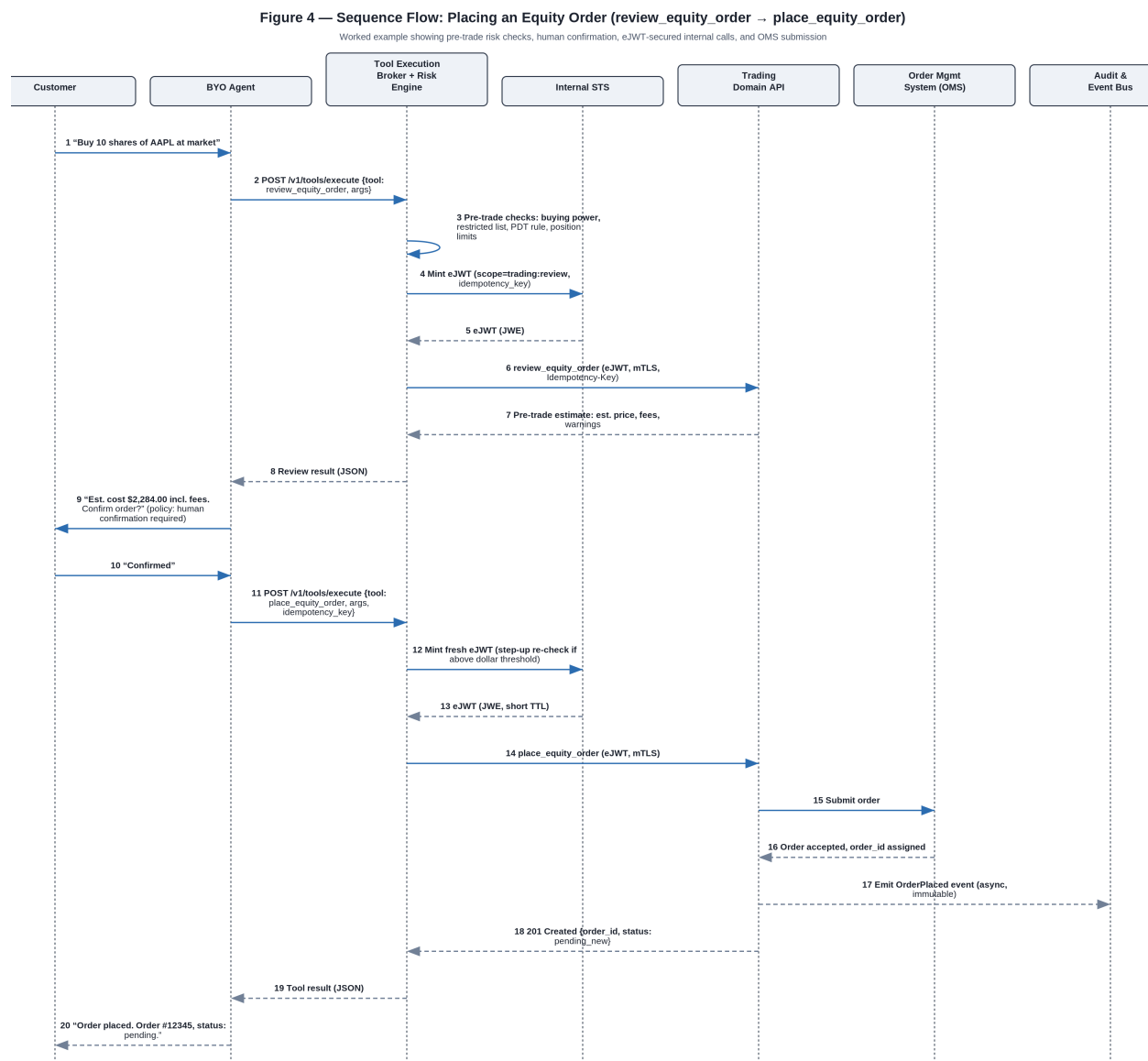


Figure 4: Sequence flow for placing an equity order: pre-trade risk checks, human-in-the-loop confirmation, eJWT-secured internal calls, and Order Management System submission with audit event emission.

Two design points worth calling out explicitly: first, `review_equity_order` and `place_equity_order` are always two distinct tool calls with two distinct eJWTs (step 4-5 and step 12-13) — a review can never silently become a placement; second, the Idempotency-Key generated by the agent at step 11 is propagated all the way to the Order Management System, so a network retry between the agent and the broker can never result in a duplicate order.

9 Trading and Research Tool Catalog (Initial Scope)

The initial catalog is deliberately close to the tool surface already proven in production agentic-trading offerings (for example, the tool set Robinhood documents for its own agentic trading product), adapted to this firm’s domain model and security controls.

9.1 Research Tools

Tool	Scope required	Description
search_symbol	research:read	Resolve a company name or partial name to a ticker symbol
get_equity_quotes	research:read	Real-time quote and prior close for up to 20 symbols
get_equity_fundamentals	research:read	Valuation ratios, market cap, 52-week range, dividend data
get_equity_historicals	research:read	OHLCV price bars over a requested time range
get_equity_technical_indicators	research:read	RSI, MACD, Bollinger Bands, moving averages, and similar indicators
get_earnings_calendar	research:read	Scheduled earnings reports across the market over a date window
get_earnings_results	research:read	Recent/upcoming earnings dates and EPS estimates for a symbol
get_index_quotes	research:read	Real-time index values
get_watchlists / get_watchlist_items	research:read	List watchlists and their contents
create_watchlist / add_to_watchlist	watchlist:write	Create and maintain custom watchlists
get_scans / run_scan	research:read	List and execute saved market scanners

9.2 Trading Tools

Tool	Scope required	Risk tier	Description
get_accounts	trading:read	LOW	List the customer’s brokerage accounts eligible for agent access
get_portfolio	trading:read	LOW	Portfolio value, allocation by asset class, buying power
get_equity_position	trading:read	LOW	Open equity positions, quantity, cost basis
get_equity_orders	trading:read	LOW	Order status history

Tool	Scope required	Risk tier	Description
get_equity_tradability	trading:read	LOW	Whether a symbol is tradable, including fractional eligibility
review_equity_order	trading:review	MEDIUM	Simulate an equity order; returns estimated price, fees, and pre-trade warnings — places no order
place_equity_order	trading:execute:equity	HIGH	Place a live equity order
cancel_equity_order	trading:execute:equity	MEDIUM	Cancel an open equity order
get_option_chains / get_option_instruments	trading:read	LOW	Load options chains and contracts by expiry/strike/type
get_option_position / get_option_orders	trading:read	LOW	Options positions and order history
review_option_order	trading:review	MEDIUM	Simulate an options order with pre-trade alerts
place_option_order	trading:execute:option	HIGH	Place a live options order (requires existing options approval on the account)
cancel_option_order	trading:execute:option	MEDIUM	Cancel an open options order

9.3 Tool Definition Schema Example: place_equity_order

```
{
  "tool_id": "place_equity_order",
  "version": "1.3.0",
  "description": "Place a new equity order. Requires a prior review_equity_order call and explicit approval on the account",
  "required_scope": "trading:execute:equity",
  "input_schema": {
    "type": "object",
    "required": ["account_id", "symbol", "side", "order_type", "quantity_or_notional", "time_in_force"],
    "properties": {
      "account_id": { "type": "string", "description": "Brokerage account ID; must be in the tool's account list" },
      "symbol": { "type": "string", "pattern": "^[A-Z]{1,10}$" },
      "side": { "type": "string", "enum": ["buy", "sell"] },
      "order_type": { "type": "string", "enum": ["market", "limit", "stop", "stop_limit"] },
      "quantity_or_notional": {
        "type": "object",
        "oneOf": [
          { "properties": { "shares": { "type": "number", "minimum": 0.0001 } }, "required": ["shares"] },
          { "properties": { "notional_usd": { "type": "number", "minimum": 1 } }, "required": ["notional_usd"] }
        ]
      }
    }
  },
}
```



```
    "limit_price": { "type": "number", "minimum": 0 },
    "time_in_force": { "type": "string", "enum": ["day", "gtc", "ioc"] },
    "idempotency_key": { "type": "string", "format": "uuid" },
    "customer_confirmation": { "type": "boolean", "description": "true only if the customer e
  }
},
"output_schema": {
  "type": "object",
  "properties": {
    "order_id": { "type": "string" },
    "status": { "type": "string", "enum": ["pending_new", "accepted", "rejected"] },
    "reject_reason": { "type": "string" }
  }
}
}
```

10 Worked Example: Placing a Trade via a Third-Party Agent

This section traces the example in Figure 4 with concrete request/response payloads, from natural language to a confirmed order.

Customer, to their own agent (e.g., Claude or ChatGPT connected to the brokerage tool endpoints):

“Buy 10 shares of AAPL at market in my brokerage account.”

Step 1 — The agent finds the right tool. Having already authenticated (Section 8.2) and cached the tool catalog from an earlier search, the agent knows `place_equity_order` requires a prior `review_equity_order` call.

Step 2 — Review call.

POST /v1/tools/execute

Authorization: DPoP <access_token>

Idempotency-Key: 3fae1d2c-9b41-4b7a-8f2e-2b8b6a2b9c11

```
{
  "tool_id": "review_equity_order",
  "arguments": {
    "account_id": "acct_9f21ac",
    "symbol": "AAPL",
    "side": "buy",
    "order_type": "market",
    "quantity_or_notional": { "shares": 10 },
    "time_in_force": "day"
  }
}
```

Response:

```
{
  "estimated_price": 228.40,
  "estimated_notional_usd": 2284.00,
  "estimated_fees_usd": 0.00,
  "buying_power_after_usd": 4116.22,
  "warnings": []
}
```

Step 3 — Human confirmation (policy-enforced, not optional for this risk tier). The agent relays the estimate to the customer and requires an explicit affirmative reply before proceeding:

Agent: “This will cost approximately \$2,284.00 including fees, leaving \$4,116.22 in buying power. Confirm?” Customer: “Yes, confirmed.”

Step 4 — Placement call, carrying a *new* idempotency key and `customer_confirmation: true`:

POST /v1/tools/execute

Authorization: DPoP <access_token>

Idempotency-Key: 7c1a90de-44b2-4e9a-9d3f-51a7c0e4a220

```
{
  "tool_id": "place_equity_order",
```

```
"arguments": {
  "account_id": "acct_9f21ac",
  "symbol": "AAPL",
  "side": "buy",
  "order_type": "market",
  "quantity_or_notional": { "shares": 10 },
  "time_in_force": "day",
  "idempotency_key": "7c1a90de-44b2-4e9a-9d3f-51a7c0e4a220",
  "customer_confirmation": true
}
}
```

Response:

```
{
  "order_id": "ord_20260730_004821",
  "status": "pending_new"
}
```

Step 5 — The agent reports back to the customer:

“Order placed — order #ord_20260730_004821, status pending. I’ll let you know when it fills.”

Internally, this single conversational exchange produced two distinct, independently authorized, independently audited tool calls, each backed by its own freshly minted eJWT (Section 7), its own idempotency key, and an immutable audit event on the internal event bus (Section 11.5) — indistinguishable, from a recordkeeping standpoint, from an order placed through the firm’s own mobile app, except for the additional act claim identifying that this order was placed via a third-party agent.

11 Security Best Practices

11.1 Zero Trust and Least Privilege

No component implicitly trusts another based on network location alone. Every hop — agent-to-edge, edge-to-broker, broker-to-domain-API — independently authenticates and authorizes the call. Scopes are the narrowest that accomplish the customer’s stated intent; a read-only research session never carries trading scopes, and a trading session’s dollar/quantity limits are enforced both at token-issuance time (RFC 9396 grant) and again at execution time (Policy/Risk Engine).

11.2 Perimeter and Transport Security

- TLS 1.3 everywhere; mutual TLS for all Zone B↔C and Zone C↔D traffic.
- WAF with brokerage-specific rule sets (credential stuffing, order-flow abuse patterns) in front of the edge gateway.
- Sender-constrained tokens (DPoP or mTLS) so a leaked bearer token cannot be replayed from a different client.
- Strict CORS/redirect-URI allow-listing tied to the RFC 7591 dynamic client registration record for each agent platform.

11.3 Anti-Automation, Abuse, and Prompt-Injection Defenses

Agents introduce a threat class distinct from traditional API abuse: a malicious or compromised *data source* the agent consults (a news article, a webpage, a document) can attempt to inject instructions that manipulate the agent into taking unintended actions (“prompt injection”). Mitigations in this architecture:

- **The Tool Execution Broker is the enforcement point, not the agent.** Scope, limit, and risk-tier checks happen server-side on every call, regardless of what the agent “believes” it is authorized to do — an injected instruction cannot grant a scope the token does not carry.
- **High-risk tools always require a fresh, explicit customer_confirmation flag** tied to the current conversational turn (Section 9.3, Section 10); the broker does not trust an agent’s self-reported claim that the customer approved an action without this structural signal, and firms may additionally require out-of-band confirmation (push notification to the firm’s own app) for orders above a configurable threshold.
- **Per-customer and per-agent-client rate limits and velocity checks** at the edge gateway and in the risk engine catch runaway or looped agent behavior (e.g., an agent stuck retrying an order) before it reaches the Order Management System.
- **Response shaping** (Section 5.5, step 6) ensures tool outputs returned to the agent never include internal system data, other customers’ data, or fields outside the published output_schema — limiting what an agent (and anything it might be tricked into exfiltrating) can access in the first place.

11.4 Human-in-the-Loop and Trade Controls

- Orders above a customer-configurable auto-confirm threshold always require explicit, current-turn confirmation (Section 9.3).
- Customers may configure their account to require confirmation for *all* trades regardless of size, or — consistent with disclosed risk, as in the Robinhood pattern reviewed in Section 3.1 — opt into unattended trading within firm-approved limits, with that election itself captured as an auditable consent event.

- A firm-wide **kill switch**, scoped per customer, per agent client, or globally, can immediately revoke all outstanding tokens and reject new PAR/token requests for the agentic channel without affecting the firm’s other channels.
- Pattern Day Trading, buying-power, restricted-security, and suitability checks run identically whether the order originates from the mobile app or an agent (Section 5.5, step 4; Figure 4 step 3).

11.5 Auditability and Non-Repudiation

- Every tool-execute call is logged at the edge gateway, the broker, and the domain API, correlated by a shared trace ID and the eJWT’s jti.
- The Audit & Event Bus (Zone D) receives an immutable event for every order lifecycle transition, tagged with the act claim identifying the originating agent platform — supporting FINRA/SEC recordkeeping obligations that must reconstruct who (or what) initiated a given order.
- Consent grants, scope changes, and step-up authentication events are themselves audited, independent of any subsequent tool call.

11.6 Secrets and Key Management

- STS signing/encryption keys live in an HSM; no private key material exists outside it (Section 7.4).
- Agent platform OAuth client credentials (where confidential clients are used) are stored in a managed secrets store with automatic rotation, never in configuration files or source control.
- JWKS endpoints are the only mechanism for key distribution; no key material is ever embedded in client applications or documentation.

11.7 Data Minimization and Response Shaping

Tool outputs are defined by an explicit `output_schema` (Section 9.3) rather than being a pass-through of the internal domain API’s response. This means adding a new internal field to the Trading Domain API never silently expands what an agent can see — a deliberate registry change is required, which is itself governed (Section 12).

11.8 Regulatory Considerations

- **FINRA/SEC recordkeeping (17a-4, Rule 3110)**: every agent-initiated order must be reconstructable with the same fidelity as an app-initiated order; the act claim and audit event bus are designed to satisfy this directly.
- **Suitability/Reg BI-adjacent controls**: the Policy/Risk Engine’s pre-trade checks are the same checks applied to the firm’s own channels, not a parallel, weaker path.
- **Regulation S-P / data privacy**: response shaping (Section 11.7) and scope minimization (Section 6.4) limit what customer financial data ever reaches a third-party agent platform’s infrastructure, which is itself outside the firm’s control once returned.
- **Third-party risk management**: agent platforms are subject to the firm’s vendor/third-party risk assessment before being added to the RFC 7591 allow-list (Section 6.6), consistent with existing third-party API consumer onboarding.

12 Governance and Lifecycle Management

- **Tool lifecycle states:** draft → beta (limited allow-listed agent clients) → GA → deprecated (still callable, flagged in responses) → retired (returns 410 Gone). State transitions require Architecture Review Board sign-off for any HIGH risk-tier tool.
- **Versioning:** tool input_schema/output_schema changes follow semantic versioning; a breaking change ships as a new minor/major version with the prior version supported through a published deprecation window, so an agent platform's cached tool definitions do not fail without notice.
- **Agent platform onboarding:** a defined intake process (security review, RFC 7591 registration, sandbox certification against a non-production tool catalog) before an agent platform's client_id is added to the production allow-list.
- **Change management:** registry changes are themselves version-controlled and deployed through the firm's standard CI/CD and change-approval process; the registry, not the underlying domain API, is the unit of agent-facing change control.

13 Non-Functional Requirements

Category	Requirement
Latency	P99 tool-execute round trip (excluding customer think-time) under 800 ms for read tools, under 1.5 s for order-placement tools
Availability	Tool Gateway and STS deployed multi-AZ, $\geq 99.95\%$ monthly availability, independent of the firm's web/mobile channel deploys
Scalability	Stateless broker and STS scale horizontally; tool search index scales independently of execution path
Observability	Distributed tracing across all four zones correlated by trace ID; SLO dashboards per tool, per agent platform
Rate limiting	Per-customer and per-agent-client limits, independently tunable per tool risk tier
Disaster recovery	RTO/RPO aligned to the firm's existing Trading Domain API tier-1 targets, since the Tool Gateway sits in the critical path for order placement

14 Roadmap

- **Phase 2 — Money Movement:** new tool namespace (`money_movement:*` scopes), mandatory step-up MFA on every transfer regardless of amount, and a lower default auto-confirm threshold than Trading.
- **Phase 2 — Account Management / Customer Management:** read-mostly initially (statements, tax documents); any identity-changing tool (e.g., updating a beneficiary) requires a dedicated, higher-assurance authentication ceremony.
- **Optional MCP adapter:** because the Tool Registry and Execution Broker are protocol-agnostic (Section 5), a thin MCP adapter could later expose the same registry over an MCP transport for agent platforms that require it, without changing the underlying identity, eJWT, or risk-engine model — MCP would become an additional transport option, not a replacement for this architecture.
- **Agentic Commerce Protocol alignment:** as delegated-payment-token standards (e.g., Shared Payment Tokens) mature industry-wide, Money Movement tools can adopt a similar scoped, single-purpose token pattern for funding an agent-initiated transfer, consistent with the eJWT/RFC 9396 approach already used for Trading.

Appendix A – Glossary

Term	Definition
BYOA	Bring-your-own-agent: an AI agent chosen and operated by the customer, not by the brokerage
DPoP	Demonstrating Proof-of-Possession — sender-constrains an OAuth token to a client-held key pair
eJWT	Encrypted JSON Web Token — a signed-then-encrypted (JWS-in-JWE) internal identity assertion minted by the internal STS
FAPI 2.0	Financial-grade API security profile built on OAuth 2.0/2.1 for high-value API scenarios
JWE / JWS	JSON Web Encryption / JSON Web Signature
MCP	Model Context Protocol — an open standard for connecting AI agents to external tools and data
PAR	Pushed Authorization Request — submits OAuth authorization parameters via a back-channel call
RAR	Rich Authorization Requests (RFC 9396) — fine-grained, structured authorization scopes
STS	Security Token Service — issues internal identity tokens (here, the eJWT issuer)
Tool Registry / Search / Broker	The three Zone C components that catalog, discover, and execute agent-facing tools

Appendix B – eJWT Claims Quick Reference

See Section 7.3 for the full table (iss, sub, act, accounts, scope, limits, aal, risk_score, idempotency_key, exp/iat/jti).

Appendix C – References

- Robinhood, “Trading with your agent” and “Agentic Trading overview” — <https://robinhood.com/us/en/s-with-your-agent/> and <https://robinhood.com/us/en/support/articles/agentic-trading-overview/>
- IETF, “OAuth 2.0 Extension: On-Behalf-Of User Authorization for AI Agents” (draft) — <https://www.ietf.org/archive/id/draft-oauth-ai-agents-on-behalf-of-user-01.html>
- RFC 8693, “OAuth 2.0 Token Exchange”
- RFC 9396, “OAuth 2.0 Rich Authorization Requests”
- RFC 7591, “OAuth 2.0 Dynamic Client Registration Protocol”
- OpenID Foundation, “FAPI 2.0 Security Profile” — <https://oauth.net/fapi/>
- NIST SP 800-63B, “Digital Identity Guidelines: Authentication and Lifecycle Management”

- Agentic Commerce Protocol (OpenAI/Stripe) — <https://github.com/agentic-commerce-protocol/agentic-commerce-protocol>